=== UNIT 5: DOM MANIPULATION & EVENT-DRIVEN PROGRAMMING ===

>> 5.1 The Document Object Model (DOM) Architecture

The DOM is the language-neutral W3C standard programming interface that represents HTML documents as a structured hierarchy of live objects, enabling scripts to dynamically manipulate document structure, styling, and content at runtime:
* The Window and Document Objects:
- window: Global root object representing the browser window or tab. Contains Browser Object Model (BOM) APIs (navigator, location, history, screen, localStorage).
- document: The root child of window representing the parsed HTML document.

>> 5.2 DOM Element Selection and Node Traversal

* Modern Selection API: document.getElementById('id'), document.querySelector('css_selector') (returns first matching element), document.querySelectorAll('css_selector') (returns static NodeList of all matching elements).
* DOM Tree Traversal Properties: .parentNode, .parentElement, .children (HTMLCollection of element children), .firstElementChild, .lastElementChild, .nextElementSibling, .previousElementSibling.

>> 5.3 Modifying Content, Attributes and Styles

* Content Modification: .textContent (safe plain text, strips HTML tags), .innerText (rendered text aware of CSS styling), .innerHTML (parses and inserts HTML markup; vulnerable to Cross-Site Scripting - XSS if used with untrusted user input).
* Attribute Manipulation: .getAttribute('attr'), .setAttribute('attr', 'value'), .removeAttribute('attr'), .hasAttribute('attr'). Dedicated property access: element.id, element.src, element.href.
* ClassList API: Best practice for styling: element.classList.add('class'), .remove('class'), .toggle('class'), .contains('class'), .replace('old', 'new').
* Direct Element Creation: const el = document.createElement('tagName'); el.textContent = '...'; parent.appendChild(el); parent.insertBefore(newEl, referenceEl); element.remove().

>> 5.4 Event-Driven Architecture and Event Listeners

JavaScript in the browser is event-driven: code executes in response to asynchronous user actions or browser events:
* The addEventListener Method: element.addEventListener(eventType, handlerFunction, options).
- Advantages: Allows multiple independent listeners on the same event; supports event capture phase; works with anonymous arrow functions.
* Common Event Types: click, dblclick, mousedown, mouseup, mouseenter, mouseleave, keydown, keyup, submit, change, input, focus, blur, scroll, resize, DOMContentLoaded.

>> 5.5 Event Propagation: Bubbling, Capturing & Event Delegation

When an event occurs on a nested child element, it propagates through three sequential phases:
* 1. Capturing (Trickling) Phase: Event travels down from window -> document -> body -> parent to the target element.
* 2. Target Phase: Event fires on the target element itself.
* 3. Bubbling Phase: Event bubbles upward from target -> parent -> body -> document -> window.
* Event Control Methods: event.stopPropagation() (halts upward bubbling), event.preventDefault() (cancels default browser action, e.g. prevents form submission or link following).
* Event Delegation Pattern: Performance optimization where a single event listener is attached to a common parent element rather than attaching hundreds of individual listeners to child elements. Uses event.target to identify which child triggered the event.

[PROGRAM 1: Dynamic Interactive Task To-Do Application]

Problem Context: Implement a complete task list manager where users type tasks, click 'Add' to dynamically generate DOM elements, click a task to toggle completion class, and click a delete button using Event Delegation.
HTML & JavaScript Code:

```
<div class='todo-app'>
<input type='text' id='taskInput' placeholder='Enter new assignment...'>
<button id='addTaskBtn'>Add Task</button>
<ul id='taskList'></ul>
</div>
```

```
<script>
const taskInput = document.getElementById('taskInput');
const addTaskBtn = document.getElementById('addTaskBtn');
const taskList = document.getElementById('taskList');

// Add Task Function
function addTask() {
const text = taskInput.value.trim();
if (!text) return alert('Task description cannot be empty.');

const li = document.createElement('li');
li.className = 'task-item';
li.innerHTML = '<span>'${text}</span> <button class='btn-delete'>&times;</button>';
taskList.appendChild(li);
taskInput.value = '';
taskInput.focus();
}

addTaskBtn.addEventListener('click', addTask);
taskInput.addEventListener('keydown', (e) => { if (e.key === 'Enter') addTask(); });

// Event Delegation on parent taskList
taskList.addEventListener('click', (e) => {
if (e.target.classList.contains('btn-delete')) {
e.target.closest('li').remove(); // Delete task item
} else if (e.target.tagName === 'SPAN' || e.target.tagName === 'LI') {
e.target.closest('li').classList.toggle('completed'); // Toggle completion
}
});
</script>
```

Key Features: createElement, innerHTML interpolation, keyboard Enter handling, and efficient Event Delegation on taskList.

[PROGRAM 2: Real-Time Form Validation with Visual Error States]

Problem Context: Implement real-time client-side form validation for username, email, and password strength, intercepting submit via event.preventDefault() and displaying dynamic error badges.

JavaScript Source Code:

```
const form = document.getElementById('signupForm');
```

```
const emailField = document.getElementById('emailField');
const passField = document.getElementById('passField');
const errorBox = document.getElementById('errorBox');

form.addEventListener('submit', (e) => {
e.preventDefault(); // Stop default form page submission
let errors = [];

const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(emailField.value)) {
errors.push('Please provide a valid email address.');
emailField.classList.add('is-invalid');
} else {
emailField.classList.remove('is-invalid');
}

if (passField.value.length < 8) {
errors.push('Password must contain at least 8 characters.');
passField.classList.add('is-invalid');
} else {
passField.classList.remove('is-invalid');
}

if (errors.length > 0) {
errorBox.innerHTML = errors.map(err => '<p class='error-text'>'${err}</p>').join('');
errorBox.classList.remove('d-none');
} else {
errorBox.classList.add('d-none');
alert('Form validated successfully! Dispatching AJAX payload.');
}
});
```

Key Techniques: Regex pattern testing, classList toggle for CSS error styling, and preventing default navigation.

[PROGRAM 3: Dynamic Theme Switcher with LocalStorage Persistence]

Problem Context: Build a dark/light mode toggle button that manipulates CSS class on the <body> tag and persists the user preference across browser restarts using localStorage.

JavaScript Source Code:

```
const themeToggleBtn = document.getElementById('themeToggleBtn');
```

```
// Initialize theme from LocalStorage on page load
const savedTheme = localStorage.getItem('user_theme') || 'light';
if (savedTheme === 'dark') {
document.body.classList.add('dark-mode');
themeToggleBtn.textContent = 'Switch to Light Mode';
}

themeToggleBtn.addEventListener('click', () => {
document.body.classList.toggle('dark-mode');
const isDark = document.body.classList.contains('dark-mode');

// Save state to LocalStorage
localStorage.setItem('user_theme', isDark ? 'dark' : 'light');
themeToggleBtn.textContent = isDark ? 'Switch to Light Mode' : 'Switch to Dark Mode';
});
```

Key API: localStorage.setItem and getItem combined with document.body.classList.toggle.

[PROGRAM 4: Modal Dialog with Keyboard Accessibility]

Problem Context: Build an accessible modal popup window that opens on button click, closes on backdrop click or close-button click, and closes when the user presses the 'Escape' key.

JavaScript Source Code:

```
const openModalBtn = document.getElementById('openModalBtn');
const closeModalBtn = document.getElementById('closeModalBtn');
const modalBackdrop = document.getElementById('modalBackdrop');

function openModal() {
modalBackdrop.classList.add('show');
modalBackdrop.setAttribute('aria-hidden', 'false');
}
function closeModal() {
modalBackdrop.classList.remove('show');
modalBackdrop.setAttribute('aria-hidden', 'true');
}

openModalBtn.addEventListener('click', openModal);
closeModalBtn.addEventListener('click', closeModal);
```

```
// Close when clicking outside modal content
modalBackdrop.addEventListener('click', (e) => {
if (e.target === modalBackdrop) closeModal();
});

// Keyboard accessibility: Close on Escape key
document.addEventListener('keydown', (e) => {
if (e.key === 'Escape' && modalBackdrop.classList.contains('show')) {
closeModal();
}
});
```

>> 5.6 University Review & Examination Questions

* Q1: Explain Event Bubbling and Event Capturing in the DOM event flow, detailing how Event Delegation exploits this architecture.
Answer: When an event occurs on a target DOM element, browser event propagation executes in three phases: (1) Capturing phase: the event trickles down from the window, document, and ancestor nodes down to the target. (2) Target phase: the event triggers on the element itself. (3) Bubbling phase: the event bubbles upward through parent ancestors back to window. Event Delegation is a high-performance design pattern that leverages bubbling: instead of binding hundreds of event listeners to individual child elements (e.g. items in a large list), a single listener is placed on the parent container (<ul>). When any child is clicked, the event bubbles up to the parent; the parent handler checks event.target to process the action, drastically reducing memory usage and automatically supporting dynamically injected child elements.
* Q2: Differentiate between 'textContent', 'innerText', and 'innerHTML' in DOM scripting, highlighting security considerations regarding XSS attacks.
Answer: 'textContent' retrieves or sets raw text content of a node and all descendants, including hidden text (<script>, <style>), without parsing HTML markup. 'innerText' returns rendered visible text, taking CSS styling and layout into account (e.g. ignores text hidden by display: none). 'innerHTML' retrieves or sets raw HTML markup inside an element, instructing the browser engine to parse strings into DOM elements. Security Concern: Using 'innerHTML' with untrusted user-supplied input creates severe Cross-Site Scripting (XSS) vulnerabilities, enabling attackers to inject malicious <script> tags or <img onerror='...'> payloads to steal cookies and session tokens. Safe applications use 'textContent' or dedicated DOM construction methods.

>> 5.7 Key Takeaways and Revision Points

* querySelector and querySelectorAll provide modern CSS-selector based DOM lookup.
* Use classList (add, remove, toggle) rather than direct inline style assignment.
* Event Delegation binds a single listener to a parent element, leveraging event bubbling.
* Never assign un-sanitized user input to innerHTML to prevent Cross-Site Scripting (XSS).